

National Level Agentic AI **Hackathon**

Problem Statement Booklet — 16 problem statements across 4 tracks, scoped for student teams to build a working agentic AI prototype in a single hackathon window.

16

PROBLEM STATEMENTS

04

TRACKS

24

HRS BUILD WINDOW

How to Read This Booklet

Each problem statement follows the same eight-part structure — Background, Objective, Key Features, Expected Solution, Suggested Technology Stack, Expected Outcomes, Possible Use Cases, and Evaluation Parameters — so every team can compare problems on equal footing.

General Notes for All Teams

- **“Agentic” is the core evaluation lens.** A single prompt-to-LLM-to-answer chatbot will score lower than a system that shows clear multi-step reasoning, tool use, or agent hand-offs — even on a simple problem.
- **Use mock/synthetic data confidently.** None of these problems require scraping restricted government systems or using real personal data.
- **Keep scope tight.** Every problem is scoped for a 4–5 person team within the hackathon window.
- **Explainability matters.** Your agent should be able to show *why* it decided something, not just output a final answer.
- **Track D is the advanced track.** These sales/revenue problems involve stateful multi-agent pipelines, external tool integration, and reasoning under uncertainty — pick these if your team wants the biggest challenge in the booklet.

Quick Reference

#	PROBLEM STATEMENT	TRACK
A1	Ayurveda & Wellness Regulation Assistant	Track A — Conversational & RAG-Powered Assistants
A2	WeatherGPT — Conversational Weather & Local Alert Assistant	Track A — Conversational & RAG-Powered Assistants
A3	Cooperative & Legal Helpdesk Chatbot	Track A — Conversational & RAG-Powered Assistants
A4	Government Scheme-Matching Assistant	Track A — Conversational & RAG-Powered Assistants
B1	Confidential Document Q&A Workbench	Track B — Multi-Agent Orchestration & Decision Support
B2	Environmental Monitoring Multi-Agent System	Track B — Multi-Agent Orchestration & Decision Support
B3	Campus/Facility Infrastructure Decision-Support Agent	Track B — Multi-Agent Orchestration & Decision Support
B4	Product Compliance Advisor Agent	Track B — Multi-Agent Orchestration & Decision Support
C1	Voice-Based Livelihood Mapping & Skilling Recommendation Agent	Track C — Perception, Voice & Document Reasoning Agents
C2	Document & Identity Consistency-Checking Agent	Track C — Perception, Voice & Document Reasoning Agents
C3	Safety Report Analysis Agent (Incident Precursor Detector)	Track C — Perception, Voice & Document Reasoning Agents
C4	Skill-Gap-to-Job Matching Agent	Track C — Perception, Voice & Document Reasoning Agents
D1	Autonomous Lead Generation & Qualification Agent	Track D — Sales, Growth & Revenue Agents (Advanced)
D2	Personalized Multi-Touch Sales Outreach Agent	Track D — Sales, Growth & Revenue Agents (Advanced)
D3	Sales Call Intelligence & Coaching Agent	Track D — Sales, Growth & Revenue Agents (Advanced)
D4	Pipeline Revenue Forecasting & Deal-Risk Agent	Track D — Sales, Growth & Revenue Agents (Advanced)



TRACK A

Conversational & RAG-Powered Assistants

Agents that answer questions by retrieving the right information and responding conversationally, in the right language.

A1 Ayurveda & Wellness Regulation Assistant

A2 WeatherGPT — Conversational Weather & Local Alert Assistant

A3 Cooperative & Legal Helpdesk Chatbot

A4 Government Scheme-Matching Assistant

Ayurveda & Wellness Regulation Assistant

Track: Conversational & RAG-Powered Assistants

BACKGROUND

Small Ayurveda manufacturers and startups struggle to understand which regulations — AYUSH licensing, labeling rules, IP protection for traditional formulations — apply to them. Legal consultation is expensive, and government documents are dense and mostly available only in English or formal Hindi.

OBJECTIVE

Develop a multilingual AI assistant that answers questions about Ayurveda product regulation, licensing, and intellectual property — grounding every answer in a cited source document, never a guess.

KEY FEATURES

- 1 Conversational Q&A over regulation/licensing/IP documents
- 2 Source citation attached to every answer
- 3 Multilingual support (minimum English + 1 Indian language)
- 4 Confidence flagging — tells the user when it isn't sure
- 5 Query classification (licensing / labeling / IP / other)
- 6 Simple chat-style interface (web or WhatsApp-style)

EXPECTED SOLUTION

- A retrieval-augmented chatbot backed by a document vector store
- A query-understanding layer that classifies intent and language
- A citation-grounded response generator (answers only from retrieved chunks)
- A translation layer for multilingual output

SUGGESTED TECHNOLOGY STACK

Backend	Python / FastAPI
Retrieval	LangChain or LlamaIndex + ChromaDB / FAISS
LLM	Open-weight (Llama 3, Mistral) or hosted API (Claude, GPT)
Translation	Google Translate API / IndicTrans2
Frontend	Streamlit / React

EXPECTED OUTCOMES

- Faster, cheaper access to regulatory clarity for small manufacturers
- Reduced dependence on paid legal consultation for basic questions
- A reusable RAG pattern applicable to other regulated domains

POSSIBLE USE CASES

- Startup founders checking licensing requirements before product launch
- Manufacturers verifying labeling compliance
- Researchers checking IP protection status for traditional formulations

EVALUATION PARAMETERS

- Answer accuracy and grounding (no hallucination)
- Citation quality and traceability
- Multilingual coverage
- UX clarity and response latency

KEY FEATURE 7 | COMPULSORY ADD-ON Not-in-my-sources fallback

Set a similarity threshold on retrieval. If the best chunk scores below it, the agent refuses to answer, states that the documents do not cover the question, and suggests a rephrase or a category to try. This enforces the no-hallucination rule in code.

WeatherGPT — Conversational Weather & Local Alert Assistant

Track: Conversational & RAG-Powered Assistants

BACKGROUND

Weather information is scattered across multiple portals, bulletins, satellite products, and forecast systems, making it hard for common users, researchers, and disaster managers to get quick, actionable insights.

OBJECTIVE

Develop an AI-powered chatbot platform, WeatherGPT, that integrates meteorological data, forecasting models, and alert systems to deliver accurate, contextual, multilingual weather intelligence through natural conversation.

KEY FEATURES

- 1 Real-time weather information retrieval
- 2 Natural language querying for forecasts
- 3 Tool-selection logic (current vs forecast vs historical data source)
- 4 Extreme weather alerts and proactive warning dissemination
- 5 Location-based forecasting and advisory generation
- 6 Multilingual support for Indian languages
- 7 Climate trend and historical weather analysis
- 8 Voice-enabled interaction for rural accessibility

EXPECTED SOLUTION

- A conversational AI platform (web or mobile) with tool-calling
- Backend integration with a weather API/data source
- An LLM-based query-understanding and tool-selection engine
- A scheduled alert mechanism for saved locations

SUGGESTED TECHNOLOGY STACK

Backend	Python / FastAPI / Node.js
Agent framework	LangChain / LangGraph (tool-calling)
Weather data	OpenWeatherMap API / WeatherAPI.com (free tier)
Notifications	Twilio / WhatsApp Cloud API
Database	PostgreSQL / MongoDB

EXPECTED OUTCOMES

- Faster dissemination of weather information
- Improved public accessibility to forecasts
- Better disaster preparedness and response at the individual level

POSSIBLE USE CASES

- Farmers seeking crop-weather advisories
- Commuters checking hyperlocal conditions
- Event planners checking multi-day forecasts
- Basic flood/heavy-rain alert dissemination

EVALUATION PARAMETERS

- Accuracy and relevance of weather data
- Quality of tool selection (does it pick the right data source?)
- Response latency
- Multilingual capability
- Working alert mechanism

KEY FEATURE 7 | COMPULSORY ADD-ON Tool-selection trace

Show a small panel under each reply listing which tool the agent picked (current, forecast, or historical), the parameters it used, and a one-line reason. It makes tool-selection quality visible to judges.

Cooperative & Legal Helpdesk Chatbot

Track: Conversational & RAG-Powered Assistants

BACKGROUND

India has hundreds of thousands of cooperative societies (dairy co-ops, credit societies, farmer producer organizations), but members — especially in rural areas — rarely understand bylaws, election procedures, or how to raise a grievance. This causes disputes and mismanagement.

OBJECTIVE

Develop a multilingual chatbot that helps cooperative society members understand governance rules, procedures, and their legal rights, and guides them step-by-step through common tasks.

KEY FEATURES

- 1 Query routing between governance/procedural and legal/rights-based questions
- 2 Retrieval-based answers from the Cooperative Societies Act and bylaws
- 3 Step-by-step guided flows for common procedures
- 4 Multilingual support
- 5 Chat-based interface with conversation memory

EXPECTED SOLUTION

- A router that classifies each query and directs it to the right knowledge base
- A RAG pipeline over cooperative law and bylaws documents
- A guided-flow (state machine) module for procedural tasks
- A multilingual response layer

SUGGESTED TECHNOLOGY STACK

Backend	Python / FastAPI
Retrieval	LangChain + ChromaDB / FAISS
Orchestration	LangGraph (for guided multi-turn flows)
LLM	Open-weight or hosted API
Frontend	Web chat widget or WhatsApp-style interface

EXPECTED OUTCOMES

- Reduced disputes caused by lack of procedural clarity
- Faster resolution of member grievances
- Improved transparency in cooperative governance

POSSIBLE USE CASES

- A member checking their voting rights before an election
- A farmer filing a grievance against management
- A new cooperative founder learning registration steps

EVALUATION PARAMETERS

- Correct routing between governance vs legal queries
- Accuracy of guided-flow steps
- Language coverage
- Usability of the guided flow

KEY FEATURE 7 | COMPULSORY ADD-ON Grievance letter generator

When a guided flow ends in a grievance, fill a letter template from the conversation state (member name, society, issue, relevant bylaw cited) and offer it as a downloadable text or PDF file.

Government Scheme-Matching Assistant

Track: Conversational & RAG-Powered Assistants

BACKGROUND

India has hundreds of government schemes for entrepreneurs, students, and farmers, but most eligible people never discover them because eligibility rules are scattered across many portals and written in dense language.

OBJECTIVE

Develop an agent that collects a short profile from a user and identifies exactly which government schemes they are eligible for, explaining the reasoning behind each match.

KEY FEATURES

- 1 Conversational profile intake (5–8 questions)
- 2 Rule-based eligibility checking against curated schemes
- 3 Reasoning shown for each match ("why you qualify")
- 4 Near-miss detection — shows what would make someone eligible
- 5 Ranking of matched schemes by relevance/benefit

EXPECTED SOLUTION

- A conversational intake form that adapts questions based on prior answers
- An eligibility-reasoning engine (rules + LLM explanation)
- A ranking and report-generation layer

SUGGESTED TECHNOLOGY STACK

Backend	Python / FastAPI
Reasoning	Rules engine (Python) + LLM for explanation
Orchestration	LangGraph
Data	Curated scheme dataset (MyScheme.gov.in, PMEGP, MUDRA, Stand-Up India, etc.)

EXPECTED OUTCOMES

- Increased scheme uptake among eligible but unaware citizens
- Reduced dependence on intermediaries/agents for scheme discovery
- A transparent, explainable eligibility-matching pattern reusable for other domains

POSSIBLE USE CASES

- A first-time entrepreneur checking loan/subsidy eligibility
- A student checking scholarship eligibility
- An NGO field worker helping community members discover schemes

EVALUATION PARAMETERS

- Correctness of eligibility logic
- Clarity of reasoning shown to the user
- Scheme coverage
- Overall UX

KEY FEATURE 7 | COMPULSORY ADD-ON Required-documents checklist

For each matched scheme, attach a short list of documents the user must have ready. Store it as a field in the scheme dataset and render it in the final report.



TRACK B

Multi-Agent Orchestration & Decision Support

Systems where multiple specialized agents collaborate, hand off tasks, and reach a combined decision.

B1 Confidential Document Q&A Workbench

B2 Environmental Monitoring Multi-Agent System

B3 Campus/Facility Infrastructure Decision-Support Agent

B4 Product Compliance Advisor Agent

Confidential Document Q&A Workbench

Track: Multi-Agent Orchestration & Decision Support

BACKGROUND

Organizations that handle sensitive information — refineries, PSUs, defence-adjacent industries — cannot send confidential documents to external cloud AI APIs, but still need AI-assisted document workflows.

OBJECTIVE

Develop a self-contained, on-premise "AI workbench" where users interact with multiple specialized agents — summarizer, Q&A, comparator — over their own uploaded documents, using only open-weight models.

KEY FEATURES

- 1 Document upload into a local knowledge base (PDF/DOCX)
- 2 Summarizer agent
- 3 Q&A agent (RAG over uploaded documents)
- 4 Comparator agent (structured diff between two documents/sections)
- 5 Orchestrator that routes requests to the right specialist agent
- 6 Fully local/offline execution (no external LLM API calls)

EXPECTED SOLUTION

- A multi-agent orchestration layer that decides which specialist(s) to invoke
- Three working specialist agents (summarizer, Q&A, comparator)
- A local vector store and locally-hosted open-weight LLM

SUGGESTED TECHNOLOGY STACK

Model hosting	Ollama (Llama 3.1 8B, Mistral 7B, Phi-3)
Orchestration	LangGraph
Vector store	ChromaDB (local)
Frontend	Streamlit

EXPECTED OUTCOMES

- A reusable in-house agentic AI pattern for confidentiality-sensitive organizations
- Reduced dependence on external cloud AI for internal document work
- Demonstration that useful agentic systems don't require paid APIs

POSSIBLE USE CASES

- Internal policy document Q&A for employees
- Comparing two versions of a contract or SOP
- Quick summarization of long internal reports

EVALUATION PARAMETERS

- Correctness of orchestration (right agent for the right request)
- Quality of each specialist agent's output
- Genuine local/offline operation
- Explainability of which agent handled what

KEY FEATURE 7 | COMPULSORY ADD-ON Local audit log

Write every request to a local file with timestamp, agent invoked, and documents touched, and expose it in a sidebar tab. It proves which agent handled what and that nothing left the machine.

Environmental Monitoring Multi-Agent System

Track: Multi-Agent Orchestration & Decision Support

BACKGROUND

Environmental monitoring usually happens in silos — air quality, water quality, and waste management are tracked by different teams with no combined risk picture for decision-makers.

OBJECTIVE

Develop a multi-agent system where specialist agents each monitor one environmental signal, and a coordinator agent combines their findings into a single area risk report.

KEY FEATURES

1	Air quality monitoring agent
2	Water quality monitoring agent
3	Waste/litter detection agent (image-based)
4	Coordinator agent that combines all signals into one risk verdict
5	Dashboard showing individual and combined findings

EXPECTED SOLUTION

- Three independent specialist agents, each processing one data type
- A coordinator agent that reasons across all three outputs
- A dashboard visualizing per-signal and combined risk

SUGGESTED TECHNOLOGY STACK

Orchestration	LangGraph / CrewAI
Vision model	YOLOv8 or MobileNet (transfer-learned) for litter detection
Data	CPCB AQI API, Kaggle water-quality datasets, Kaggle/Roboflow litter datasets
Frontend	Streamlit / React + Plotly

EXPECTED OUTCOMES

- A combined, reasoned environmental risk picture instead of siloed readings
- Faster identification of at-risk areas
- A reusable multi-agent coordination pattern for other monitoring domains

POSSIBLE USE CASES

- Municipal bodies prioritizing cleanup/inspection efforts
- Citizens checking a combined environmental risk score for their area
- Researchers tracking multi-signal environmental trends

EVALUATION PARAMETERS

- Independence and correctness of each specialist agent
- Quality of the coordinator's combined reasoning
- Dashboard clarity
- Genuine multi-agent hand-off (not just three features in one script)

KEY FEATURE 7 | COMPULSORY ADD-ON Conflict and freshness flag

The coordinator shows each signal's weight in the final verdict, warns when specialists disagree (for example air is high risk, water is low), and flags any signal whose data is stale or missing.

Campus/Facility Infrastructure Decision-Support Agent

Track: Multi-Agent Orchestration & Decision Support

BACKGROUND

When equipment (AC units, generators, elevators) breaks down, facility managers often rely on memory instead of reviewing what happened with similar equipment before — wasting time and missing recurring patterns.

OBJECTIVE

Develop an AI agent that helps facility managers diagnose new issues by retrieving similar past maintenance cases and recommending a likely cause and fix, with clear reasoning.

KEY FEATURES

- 1 Searchable knowledge base of past maintenance records
- 2 Similar-case retrieval for new complaints
- 3 Diagnosis reasoning based on retrieved cases
- 4 Fix recommendation with estimated cost/time and urgency
- 5 Plain-language explanation of the reasoning chain

EXPECTED SOLUTION

- A semantic search layer over historical maintenance logs
- A diagnosis agent that reasons over retrieved similar cases
- A recommendation agent for fix steps and urgency
- A simple logging/dashboard interface

SUGGESTED TECHNOLOGY STACK

Retrieval	FAISS / ChromaDB (semantic similarity search)
Reasoning	LLM + LangGraph (retrieval → diagnosis → recommendation pipeline)
Data	Synthetic dataset of 200–300 maintenance records
Frontend	Simple web dashboard

EXPECTED OUTCOMES

- Faster, more consistent diagnosis of recurring equipment issues
- Reduced downtime through pattern-based recommendations
- An explainable decision-support pattern for maintenance teams

POSSIBLE USE CASES

- A campus facilities team triaging a new AC failure
- A hospital diagnosing repeated equipment issues
- A factory logging and prioritizing maintenance requests

EVALUATION PARAMETERS

- Relevance of retrieved similar cases
- Soundness of diagnosis reasoning
- Usability of the dashboard
- Handling of multiple simultaneous complaints (if implemented)

KEY FEATURE 7 | COMPULSORY ADD-ON Technician feedback loop

Add correct/incorrect buttons on each diagnosis. Confirmed cases are appended to the knowledge base with their resolution, so retrieval improves during the demo.

Product Compliance Advisor Agent

Track: Multi-Agent Orchestration & Decision Support

BACKGROUND

Small manufacturers often don't know which BIS (Indian Standards) certification applies to their product, leading to non-compliance and penalties. The BIS portal is hard to search without already knowing the exact standard code.

OBJECTIVE

Develop a conversational agent that asks clarifying questions about a product and recommends the applicable Indian Standard(s), along with next steps for certification.

KEY FEATURES

- 1 Adaptive clarifying-question flow based on product description
- 2 Search over a curated standards database
- 3 Ranked standard recommendation with explanation
- 4 Compliance summary report with next steps

EXPECTED SOLUTION

- A multi-turn clarification agent that narrows down product category
- A search/recommendation layer over a curated standards database
- A report generator summarizing applicable standards and next steps

SUGGESTED TECHNOLOGY STACK

Orchestration	LangGraph (multi-turn clarification loop)
Search	Keyword + semantic search over a curated standards dataset
LLM	For question generation and report writing
Data	Curated set of 30–50 common BIS standards (electronics, toys, food packaging, textiles)

EXPECTED OUTCOMES

- Faster identification of applicable compliance standards for small manufacturers
- Reduced non-compliance and associated penalties
- A reusable clarification-driven recommendation pattern

POSSIBLE USE CASES

- A toy manufacturer identifying required safety certification
- A food packaging startup checking applicable standards
- An electronics maker verifying certification requirements before launch

EVALUATION PARAMETERS

- Quality of clarifying questions (do they genuinely narrow down the category?)
- Accuracy of standard recommendation
- Clarity of the final compliance report

KEY FEATURE 7 | COMPULSORY ADD-ON Question budget with early exit

Cap clarifying questions at four and stop asking as soon as the top standard leads the runner-up by a set score margin. The agent then explains why it stopped asking.



TRACK C

Perception, Voice & Document Reasoning Agents

Agents that start from unstructured input (voice, images, free-text documents) and turn it into structured action.

C1 Voice-Based Livelihood Mapping & Skilling Recommendation Agent

C2 Document & Identity Consistency-Checking Agent

C3 Safety Report Analysis Agent (Incident Precursor Detector)

C4 Skill-Gap-to-Job Matching Agent

Voice-Based Livelihood Mapping & Skilling Recommendation Agent

Track: Perception, Voice & Document Reasoning Agents

BACKGROUND

Many rural workers cannot fill out long, text-based skilling-program forms, especially in English. A natural voice conversation in their own language would remove this barrier to accessing skilling programs.

OBJECTIVE

Develop a voice-first agent that converses with a person about their current work and skills in their own language, maps this to structured skill categories, and recommends relevant government skilling programs.

KEY FEATURES

- 1 Voice input in at least one Indian language
- 2 Natural conversational flow to understand occupation and experience
- 3 Mapping of free-text responses to structured skill/NSQF categories
- 4 Skilling program recommendation with reasoning

EXPECTED SOLUTION

- A speech-to-text pipeline for voice input
- A conversational agent that asks natural follow-up questions
- A skill-mapping layer (free text → structured categories)
- A recommendation engine matching mapped skills to programs

SUGGESTED TECHNOLOGY STACK

Speech-to-text	Whisper (open-source) / Google Speech-to-Text
Conversation	LLM-based dialogue agent
Mapping	Embedding similarity or LLM classification
Data	Simplified NSQF category list (20–30 categories), 15–20 curated skilling programs
Text-to-speech (optional)	For a full voice loop

EXPECTED OUTCOMES

- Improved accessibility of skilling programs for non-literate or non-English-speaking users
- Higher enrollment in relevant, well-matched training programs
- A reusable voice-first intake pattern for other government services

POSSIBLE USE CASES

- A rural worker discovering a relevant Skill India program by voice
- A field NGO worker helping community members enroll
- A basic IVR-style skilling helpline

EVALUATION PARAMETERS

- Transcription accuracy
- Naturalness and effectiveness of the conversation
- Accuracy of skill-category mapping
- Relevance of final recommendations

KEY FEATURE 7 | COMPULSORY ADD-ON Read-back confirmation

Before recommending programs, the agent repeats the occupation and skills it understood in the user's own language and asks for a yes or no. A no loops back to a follow-up question, which catches transcription and mapping errors.

Document & Identity Consistency-Checking Agent

Track: Perception, Voice & Document Reasoning Agents

BACKGROUND

Manual document verification (for admissions, loan applications, scheme enrollment) is slow and error-prone. A pre-screening agent that flags obvious inconsistencies can save reviewers significant time — as a screening aid, not a final verdict.

OBJECTIVE

Develop an agent that checks a set of uploaded documents for internal consistency (name, date, address matches) and produces a flagged report for human review.

KEY FEATURES

- 1 OCR extraction of key fields from uploaded documents
- 2 Text normalization (handles spelling variants, date formats, abbreviations)
- 3 Cross-document consistency checking
- 4 Flagged report with confidence score
- 5 Clear disclaimer that final judgment remains human

EXPECTED SOLUTION

- An OCR pipeline for structured field extraction
- A normalization layer to reduce false-positive mismatches
- A consistency-checking engine across documents
- A human-readable screening report

SUGGESTED TECHNOLOGY STACK

OCR	Tesseract / EasyOCR
Matching	Fuzzy string matching (e.g., Levenshtein distance)
Reasoning/reporting	LLM for report generation
Data	Self-created mock/sample documents only — never real personal data

EXPECTED OUTCOMES

- Faster pre-screening of document sets before human review
- Reduced manual workload for verification teams
- A reusable consistency-checking pattern for enrollment/verification workflows

POSSIBLE USE CASES

- Pre-screening documents for scheme enrollment
- Admissions document verification support
- Loan application document pre-checks

EVALUATION PARAMETERS

- OCR extraction accuracy
- Quality of consistency logic (correctly flags real mismatches, doesn't over-flag minor variants)
- Clarity of the final report

Note: This is a screening-aid tool, not a fraud-detection verdict system. All demos must use synthetic/mock documents.

KEY FEATURE 7 | COMPULSORY ADD-ON Evidence view per flag

For every flagged mismatch, display the extracted values side by side with the source document name and the raw OCR line, so a reviewer can verify in seconds.

Safety Report Analysis Agent (Incident Precursor Detector)

Track: Perception, Voice & Document Reasoning Agents

BACKGROUND

Organizations collect thousands of free-text near-miss and safety observation reports. Buried within them are early warning patterns of serious incidents, but there is rarely time to read them all manually.

OBJECTIVE

Develop an agent that reads free-text safety reports, classifies their risk level, extracts specific risk factors, and surfaces recurring patterns across reports.

KEY FEATURES

- 1 Free-text report ingestion and parsing
- 2 Risk-level classification (low/medium/high)
- 3 Extraction of specific risk factors per report
- 4 Cross-report pattern detection (recurring themes, locations)
- 5 Dashboard summarizing trends for a safety officer

EXPECTED SOLUTION

- A report-parsing agent that extracts structured risk factors from free text
- A risk-scoring agent
- A pattern-detection layer across the full report set
- A summary dashboard

SUGGESTED TECHNOLOGY STACK

NLP/classification	LLM-based few-shot extraction and classification (no fine-tuning required)
Analysis	Pandas for trend analysis
Frontend	Streamlit / Plotly dashboard
Data	Public OSHA/Kaggle near-miss datasets, or synthetic LLM-generated reports (clearly labeled as synthetic)

EXPECTED OUTCOMES

- Earlier identification of incident precursors before they escalate
- Reduced manual review workload for safety officers
- Actionable, location/department-level safety insights

POSSIBLE USE CASES

- Factory safety officers reviewing daily near-miss reports
- Construction site safety monitoring
- Lab safety compliance tracking

EVALUATION PARAMETERS

- Quality of risk extraction and classification
- Usefulness of cross-report pattern detection
- Dashboard clarity

KEY FEATURE 7 | COMPULSORY ADD-ON Risk override with logged reason

A safety officer can change any risk label and type a reason. Store the correction and inject the latest corrections as few-shot examples in later classification calls.

Skill-Gap-to-Job Matching Agent

Track: Perception, Voice & Document Reasoning Agents

BACKGROUND

Job seekers often don't know what skills they're missing for jobs actually available near them. Generic training advice doesn't help — recommendations need to be tied to real, local job openings.

OBJECTIVE

Develop an agent that compares a person's current skills against local job requirements, identifies specific gaps, and recommends a concrete training path to close them.

KEY FEATURES

1	Profile intake (education, current skills, location, interests)
2	Matching against a curated set of job postings
3	Gap analysis — specific missing skills per job
4	Training recommendation ranked by opportunity unlocked

EXPECTED SOLUTION

- A profile-parsing agent that extracts structured skills from free text/ resume
- A job-matching agent computing a match score per posting
- A gap-analysis agent identifying missing skills for promising jobs
- A training-recommendation agent mapping gaps to real courses

EXPECTED OUTCOMES

- More targeted, actionable training recommendations
- Improved job-readiness for skill-gap candidates
- A reusable gap-analysis pattern for career guidance platforms

EVALUATION PARAMETERS

- Accuracy of skill matching
- Specificity and usefulness of gap analysis
- Quality of training recommendations and ROI reasoning

SUGGESTED TECHNOLOGY STACK

Matching	Sentence-transformers (embedding-based skill matching)
Reasoning	LLM for gap explanation and ranking
Orchestration	LangGraph (profile → match → gap → recommend pipeline)
Data	30–50 curated job postings, public course catalogs (Coursera, NPTEL, Skill India)

POSSIBLE USE CASES

- A job seeker finding out exactly what course would unlock more local job matches
- A career counseling center supporting multiple candidates
- A skilling program prioritizing which courses to offer based on regional demand

KEY FEATURE 7 | COMPULSORY ADD-ON

Time-to-ready estimate

Add course duration and cost to the course data, then show total weeks and cost to close the gaps for each target job, with a free-only filter.



TRACK D

Sales, Growth & Revenue Agents (Advanced)

Multi-agent systems that prospect, personalize, listen, and forecast. These are the hardest problems in the booklet — they demand reasoning under uncertainty, real business-tool integration, and agents that must earn trust before a human hands them outreach or revenue decisions.

D1 Autonomous Lead Generation & Qualification Agent

D2 Personalized Multi-Touch Sales Outreach Agent

D3 Sales Call Intelligence & Coaching Agent

D4 Pipeline Revenue Forecasting & Deal-Risk Agent

Autonomous Lead Generation & Qualification Agent

Track: Sales, Growth & Revenue Agents (Advanced)

BACKGROUND

Sales teams spend enormous time manually finding and qualifying leads across LinkedIn, company websites, and directories. Most of this work is repetitive pattern-matching against an Ideal Customer Profile (ICP) — exactly the kind of multi-step, tool-using task an agent should handle end to end.

OBJECTIVE

Build a multi-agent system that takes an ICP definition, discovers matching companies/contacts from public sources, enriches each lead with firmographic data, scores and ranks them against the ICP with visible reasoning, and outputs a sales-ready, de-duplicated list.

KEY FEATURES

- 1 ICP intake (industry, company size, geography, signals of interest)
- 2 Prospect-discovery agent that searches public sources for candidate companies/contacts
- 3 Enrichment agent that adds firmographic data (size, industry, tech stack, funding/hiring signals)
- 4 Qualification/scoring agent that ranks each lead against the ICP with explicit reasoning
- 5 De-duplication and CRM-ready export (CSV / HubSpot-style format)
- 6 Re-runnable batch refresh so the list can be updated periodically

EXPECTED SOLUTION

- A discovery → enrichment → scoring → export agent pipeline with clear hand-offs
- Caching/rate-limit handling so repeated runs don't hammer data sources
- A scoring rubric the agent applies consistently and explains per lead

SUGGESTED TECHNOLOGY STACK

Backend	Python / FastAPI
Orchestration	LangGraph
Discovery/Search	Serper API / SerpAPI / public company directories
Enrichment	Public company data sources (Crunchbase-style public pages, company websites) — no paid data purchase required for the demo
Data handling	Pandas for dedup/scoring, CSV/HubSpot export format

EXPECTED OUTCOMES

- Significant reduction in manual prospecting hours for a sales team
- More consistent lead qualification than ad-hoc manual judgment
- A reusable discovery-enrichment-scoring agent pattern applicable beyond sales (e.g., vendor sourcing, partnership scouting)

POSSIBLE USE CASES

- A B2B SaaS sales team building a target account list for a new region
- An agency prospecting on behalf of multiple clients
- A founder building an initial outbound list pre-seed/pre-Series A

EVALUATION PARAMETERS

- Relevance of discovered leads against the stated ICP
- Accuracy and usefulness of enrichment data
- Quality and transparency of scoring reasoning
- Correctness of de-duplication and export format

Note: Only search and use publicly available business information. Do not scrape sites that disallow it, and never collect or expose private personal data. Use mock/sample company data for the live demo if source access is uncertain on hackathon day.

KEY FEATURE 7 | COMPULSORY ADD-ON robots.txt compliance check

Before fetching any page, the discovery step checks robots.txt with Python's urllib.robotparser, skips disallowed URLs, and logs the skip. This directly satisfies the scraping note above.

Personalized Multi-Touch Sales Outreach Agent

Track: Sales, Growth & Revenue Agents (Advanced)

BACKGROUND

Generic mass outreach gets ignored. Sales reps know personalized, well-timed messages perform far better, but manually researching and writing a personalized sequence for every lead doesn't scale past a handful of accounts a day.

OBJECTIVE

Build an agent that researches each lead, drafts a personalized multi-touch outreach sequence (e.g., email + LinkedIn message across 3–4 touches), and adapts the sequence based on how the lead responds — continuing, escalating to a human, or stopping.

KEY FEATURES

- 1 Per-lead research agent that pulls recent, relevant public context (news, role, company activity)
- 2 Personalized message-drafting agent that references specific research points (not generic templates)
- 3 Multi-touch sequence planner with defined cadence and escalating intent per touch
- 4 Reply-classification agent (interested / not interested / objection / out-of-office / no reply)
- 5 Adaptive next-step agent that decides: continue sequence, pause, escalate to a human rep, or stop

EXPECTED SOLUTION

- A stateful, per-lead pipeline: research → draft → send (simulated) → classify reply → decide next step
- Persistent state per lead so the agent remembers where each conversation stands
- A clear, visible decision log showing why the agent chose each next action

SUGGESTED TECHNOLOGY STACK

Orchestration	LangGraph (stateful per-lead workflow)
Research	Web search tool + LLM summarization
Messaging (sandboxed)	SendGrid/Gmail API in test mode, or a simulated inbox — never real unsolicited sends
State/storage	PostgreSQL or SQLite for per-lead sequence state
Classification	LLM-based reply-intent classifier

EXPECTED OUTCOMES

- Higher-quality, better-personalized outreach at a volume no individual rep could sustain manually
- Faster, more consistent lead follow-up with nothing falling through the cracks
- Clear human-escalation points so the agent augments, rather than replaces, the sales rep's judgment

POSSIBLE USE CASES

- An SDR team automating first-touch outreach while staying personalized
- A small agency running outbound campaigns for multiple clients at once
- A founder doing early customer-development outreach

EVALUATION PARAMETERS

- Personalization quality — does each message actually use the researched context, not a generic template?
- Soundness of the multi-touch sequence logic and timing
- Accuracy of reply classification
- Appropriateness of escalation-to-human decisions

Note: This is a hard problem because of state management across multiple touches and the risk of feeling spammy if personalization is shallow. For the demo, use sandbox email accounts and synthetic/team-member leads — never send unsolicited outreach to real external contacts.

KEY FEATURE 7 | COMPULSORY ADD-ON Human approval queue with send guardrails

Drafts land in an approval queue instead of sending automatically. Add a suppression list and a daily send cap, so no message goes out without a rep's sign-off.

Sales Call Intelligence & Coaching Agent

Track: Sales, Growth & Revenue Agents (Advanced)

BACKGROUND

Sales managers cannot review every call their team makes, so valuable coaching moments, competitor mentions, and early deal-risk signals get missed. Reviewing calls manually also doesn't scale as a team grows.

OBJECTIVE

Build an agent that processes sales call transcripts, extracts key moments (objections, pricing discussion, competitor mentions, next steps), scores overall call quality, and generates specific, actionable coaching feedback for the rep — plus a deal-risk flag where relevant.

KEY FEATURES

1	Transcript ingestion (from provided text transcripts, or audio via speech-to-text)
2	Moment-extraction agent identifying objections, pricing talk, competitor mentions, and next-step commitments
3	Talk-ratio and filler-word/pace analysis
4	Coaching-feedback agent generating specific, timestamped, actionable feedback tied to real moments in the call
5	Deal-risk flag based on signals like unclear budget/authority/need/timeline (BANT-style gaps) or hesitation language

EXPECTED SOLUTION

- A parsing agent that structures the raw transcript into moments and speaker turns
- An analysis agent layer for both quantitative (talk ratio) and qualitative (sentiment, objections) signals
- A coaching-report generator that ties every piece of feedback to a specific transcript moment, not a vague generality

SUGGESTED TECHNOLOGY STACK

Speech-to-text (if audio)	Whisper (open-source)
Extraction/classification	LLM few-shot extraction (no fine-tuning required)
Orchestration	LangGraph
Analysis	Pandas for talk-ratio/pace metrics
Frontend	Streamlit / Plotly dashboard per call and per rep

EXPECTED OUTCOMES

- Consistent, scalable coaching feedback for every call, not just the few a manager has time to review
- Earlier detection of deals at risk due to unresolved objections or missing stakeholders
- A searchable library of coaching insights across the whole sales team

POSSIBLE USE CASES

- A SaaS sales team scaling coaching without scaling management headcount
- A call center running consistent QA across hundreds of calls a week
- A sales enablement lead building a training library from real call moments

EVALUATION PARAMETERS

- Accuracy of moment extraction (objections, competitor mentions, commitments)
- Specificity and actionability of the coaching feedback (not generic advice)
- Correctness and usefulness of deal-risk flags
- Dashboard clarity for both reps and managers

Note: Use publicly available sales-call datasets or team-recorded roleplay calls with consent. Never process real customer call recordings without explicit consent and appropriate data handling.

KEY FEATURE 7 | COMPULSORY ADD-ON Follow-up email draft

From the extracted next-step commitments, the agent drafts a recap email to the prospect that a rep can copy or edit. It reuses data already extracted.

Pipeline Revenue Forecasting & Deal-Risk Agent

Track: Sales, Growth & Revenue Agents (Advanced)

BACKGROUND

Sales forecasts are often built on rep gut-feeling rather than evidence, and deals frequently slip without anyone catching the early warning signs — a stall in activity, a disappeared stakeholder, a pushed close date — in time to intervene.

OBJECTIVE

Build an agent that analyzes CRM-style pipeline data (deal stage, amount, close date, activity history, stakeholder engagement) to produce a probabilistic revenue forecast and a ranked list of at-risk deals, each with a clear reason and a recommended next action.

KEY FEATURES

1	Ingestion of structured deal data (stage, amount, close date, last activity date, engagement history)
2	Risk-scoring agent that flags stalled deals, engagement drop-off, and missing key stakeholders
3	Forecast agent producing a probabilistic revenue forecast (weighted pipeline) with a confidence range, not a single point guess
4	Next-best-action agent recommending a specific action per at-risk deal (e.g., "re-engage economic buyer", "send updated proposal")
5	Leadership dashboard summarizing forecast, risk deals, and recommended actions

EXPECTED SOLUTION

- A statistical layer (weighted pipeline, historical win-rate by stage) feeding into an LLM reasoning layer for explanation and recommendations
- A risk-detection agent that combines rule-based signals (days since last activity, stage duration vs historical average) with LLM judgment on notes/context
- A forecast agent that shows its reasoning and confidence range, not just a number

EXPECTED OUTCOMES

- More accurate, evidence-based revenue forecasts than manual rep estimates
- Earlier, more specific intervention on deals that would otherwise silently slip
- A transparent, explainable forecasting pattern sales leadership can actually trust and act on

EVALUATION PARAMETERS

- Reasonableness of the forecast against a historical/held-out baseline
- Specificity and correctness of risk explanations (not vague "this deal looks risky")
- Usefulness of the recommended next actions
- Dashboard clarity for a non-technical sales leadership audience

SUGGESTED TECHNOLOGY STACK

Backend/analysis	Python / Pandas for pipeline statistics
Reasoning	LLM for risk explanation and next-action recommendations
Orchestration	LangGraph
Data	Synthetic CRM dataset of 100–200 deals built by the team, or a public Kaggle sales-pipeline dataset
Frontend	Streamlit / Plotly dashboard

POSSIBLE USE CASES

- Sales operations preparing a weekly pipeline review for leadership
- A sales manager triaging which deals need personal attention this week
- A RevOps team building a more disciplined forecasting process for a growing team

KEY FEATURE 7 | COMPULSORY ADD-ON

Backtest against rep estimates

Hold out one past quarter in the synthetic data, run the forecast on it, and show error against actual revenue next to the reps' own estimates. This gives the reasonableness criterion a number.